ADITYA UNIVERSITY

SCHOOL OF ENGINEERING

Department of Data Science

CERTIFICATE

This is to certify that the Cornerstone project work entitled "**BUS TICKET BOOKING SYSTEM**" is submitted by

P PRASANTHI	**(24B11DS172)**
Y HEMALATHA	**(24B11DS230)**
D SUREKHA	**(24B11DS239)**

in partial fulfillment of the requirements for the award of the B.Tech. degree in Information Technology for the academic year 2025-2026.

Project Guide	**Head of the Department**
Dr. K. Sreenu,	Dr. M. Rajababu, Ph.D.,
Assistant Professor	Associate Professor,
Department of DS	Department of DS

DECLARATION

We hereby declare that the Cornerstone project entitled "**BUS TICKET BOOKING SYSTEM**" is original and is submitted to **ADITYA UNIVERSITY**, Surampalem, in partial fulfillment of the requirements for the award of the **B.Tech. degree** in **Data science**. We further declare that this project work has not been submitted, either full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy.

Place:
Surampalem
Date:
4/4/2026

BY

P PRASANTHI	**(24B11DS172)**
Y HEMALATHA	**(24B11DS230)**
D SUREKHA	**(24B11DS239)**

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Dr. K. Sreenu, Assistant Professor**, who has guided us throughout the project. His valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. A. Lakshmana Rao, Associate Professor and Mrs. S. Neelima, Assistant Professor**, Department of IT, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department, **Dr. M. Rajababu, Associate Professor,** for his inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Engineering,** for their encouragement and support during our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

Table of contents

CONTENTS	PAGE NO
ABSTRACT	i
LIST OF FIGURES	ii
LIST OF TABLES	iii
ACRONYMS	iv
1. INTRODUCTION	1
1.1 Brief Information about the Project	1
1.2 Motivation and Contribution of Project	1
1.3 Objectives of the Project	1
1.4 Scope of the Project	2
2. SYSTEM ANALYSIS	3
2.1 Functional Requirements	3
2.2 Non-Functional Requirements	5
2.3 Requirements Specification	5
2.3.1 Minimum Hardware Requirements	5
2.3.2 Software Requirements	5
3. TECHNOLOGY DESCRIPTION	6
3.1 Programming Language	6
3.1.1 Introduction to Java	6
3.2 Framework / Libraries	6
3.2.1 Introduction to Swing	6
3.3 Database Technology	6
3.3.1 JDBC (Java Database Connectivity)	7
3.3.2 Oracle Database	7
3.4 Tools & IDEs	7
3.4.1 Visual Studio Code	7
4. DATABASE DESIGN	8
4.1 Database Tables	8
4.2 Table Structure	8
4.3 Keys and Constraints	9
4.4 Table Relationships	10
5. SYSTEM DESIGN	11
5.1 System Architecture	11
5.2 Modules Description	12

6. SYSTEM IMPLEMENTATION 13
6. CONCLUSION 12
7. FUTURE ENHANCEMENTS 13
REFERENCES 14
APPENDIX (Sample Code) 15
APPENDIX (SQL Queries) 16

ABSTRACT

The Bus Ticket Booking System is a console-based application designed to automate and simplify the process of bus ticket reservations. The system provides an efficient platform for both administrators and customers to perform various operations such as viewing available buses, booking tickets, updating reservations, and managing user information.

The application is developed using Core Java as the programming language, JDBC (Java Database Connectivity) for establishing communication between the application and the database, and Oracle Database for storing and managing data. The system follows a structured approach to handle operations like user registration, login authentication, ticket booking, cancellation, and viewing booking details.

This system helps in reducing manual work involved in traditional ticket booking methods. It minimizes errors such as double booking of seats and ensures better data accuracy and consistency. The use of database connectivity enables secure storage and quick retrieval of information.

Overall, the Bus Ticket Booking System provides a reliable, user-friendly, and efficient solution for managing transportation booking services, making it suitable for small to medium-scale applications.

LIST OF FIGURES

Figure no.	Figure Title	Page no.
1	Login Menu	13
2	Admin Menu	14
3	Customer Menu	14

LIST OF TABLES

Table no.	Table Title	Page no.
1	Users Table	8
2	Bus Table	8
3	Booking Table	9

ACRONYMS

- **JDBC** – Java Database Connectivity
- **DBMS** – Database Management System
- **SQL** – Structured Query Language
- **UI** – User Interface

1.INTRODUCTION

1.1 Brief Information about the Project

The **Bus Ticket Booking System** is a console-based application developed to simplify and automate the process of booking bus tickets. This system allows users to perform various operations such as viewing available buses, booking tickets, cancelling reservations, and checking booking details through a command-line interface.

The system is developed using **Java programming language**, and **JDBC (Java Database Connectivity)** is used to connect the application with a relational database such as **Oracle or MySQL**. The database stores all the necessary information, including bus details, passenger data, the main objective of this project is to provide a simple, efficient, and reliable ticket booking system that reduces manual effort and improves accuracy.

1.2 Motivation and Contribution of Project

In many small-scale transport systems, bus ticket bookings are still handled manually using paper records or basic methods. This leads to several issues such as long waiting times, difficulty in managing data, chances of human errors, and lack of proper record maintenance. These challenges motivated the development of the Bus Ticket Booking System

This system solves these problems

- **Automation of Ticket Booking**
- **Efficient Data Management**
- **Reduction of Errors**
- **Time Saving** Overall, this project contributes to improving the efficiency of ticket booking processes and serves as a foundation for developing more advanced systems in the future.

1.3 Objectives of the Project

- Developing an easy booking system
- To store booking details in database
- Allowing ticket booking & cancellation

1.4 Scope of the Project

The **Bus Ticket Booking System** is a console-based application designed to manage bus ticket reservations efficiently using a command-line interface. The scope of this project includes providing basic functionalities for both **admin and customer users** to perform booking-related operations through a simple and user-friendly system.

The system allows **customers** to register, log in, view available buses, book tickets, check their booking details, and cancel reservations. It ensures that seat availability is verified before confirming any booking, thereby preventing duplicate seat allocation.

On the other hand, the **admin** can manage the system by adding new buses, viewing all bookings, updating booking details, and monitoring registered users. This helps in maintaining proper control and organization of the booking system.

The project uses Java with JDBC connectivity to interact with an Oracle database, where all information related to users, buses, and bookings is stored and managed efficiently. The system ensures quick data retrieval, accuracy, and reduced manual effort.

Since the application is console-based, it requires minimal system resources and is suitable for small-scale implementations such as local travel agencies or academic purposes. It provides a strong foundation for understanding database-driven applications.

However, the system is limited to basic features and does not include advanced functionalities such as online payment, graphical user interface, or real-time tracking. These features can be added in the future to enhance the system and expand its usability.

2. SYSTEM ANALYSIS

2.1 Functional Requirements

Functional Requirements

Functional requirements define the core operations and functionalities that the **Bus Ticket Booking System (BTBS)** must perform effectively manage bus reservations and user-related information. The functional requirements of the Bus Ticket Booking System are:

• Admin Login

The system should allow the administrator to log in using valid credentials. Only authorized admin users should be able to access administrative functionalities such as managing buses, users, and bookings.

• Customer Registration

The system should allow new users to register by providing details such as user ID, username, password, email, and phone number. Proper validation should be applied to ensure that all required fields are correctly filled.

• Customer Login

The system should provide login functionality for registered customers. Users must enter valid username and password to access booking features and personal details.

• Add Bus Details

The system should allow the administrator to add new bus records, including bus ID, bus name, source, destination, total seats, and ticket price. Proper validation should be applied to ensure data accuracy.

• View Bus Details

The system should display all available bus information in a structured format. Users should be able to view details such as bus routes, available seats, and ticket prices.

• Book Ticket

The system should allow customers to book tickets by selecting bus ID, travel date, and seat number. It should calculate the total amount based on ticket price and confirm booking after successful validation.

• Seat Availability Check: Before booking a ticket, the system should check whether the selected seat is already booked for the given bus and date. If the seat is unavailable, the system should prompt the user to choose another seat.

• View Bookings

The system should allow both admin and customers to view booking details. Customers can view their own bookings, while the admin can view all bookings in the system.

• Update Booking

The system should allow the administrator to update booking details such as seat number. Proper validation should be applied to avoid conflicts.

• Cancel Booking

The system should allow customers to cancel their bookings by providing booking ID. A confirmation mechanism should be included to prevent accidental cancellation.

• View User Details

The system should allow customers to view their personal details such as name, email, and phone number stored in the database.

• Manage Users

The system should allow the administrator to view all registered users and their roles (Admin/Customer) for effective system management.

2.2 Non-Functional Requirements

• **Performance**

The system should respond quickly to operations such as login, booking ticket set

• **Security**

Only authorized users should be able to access the system using proper authentication

• **Usability**

The system should be simple and user-friendly, allowing users to perform operations

• **Reliability**

The system should function consistently without failure.

• **Scalability**

The system should be capable of handling an increasing number of users, buses, and bookings without affecting performance.

2.3 Requirements Specification

This section specifies the hardware and software needed to run the Bus Ticket Booking System.

2.3.1 Minimum Hardware Requirements

• **Processor:** Intel i3 or above
• **RAM:** 4 GB (minimum)
• **Hard Disk:** 20 GB free space
• **Monitor:** Standard display
• Keyboard and Mouse

2.3.2 Software Requirements

•**Operating System**: Windows 7/10/11
• **Programming Language**: Java
• **IDE:** VS Code, Database: Oracle Database
• **Connectivity:** JDBC

3. TECHNOLOGY DESCRIPTION

3.1 Programming Language

3.1.1 Introduction to Java

Java is a high-level, object-oriented programming language developed by Sun Microsystems. It is widely used for developing desktop, web, and enterprise applications due to its platform independence, security, and robustness.

In the Bus Ticket Booking System, Java is used to implement the core functionality of the application. It handles operations such as user login, customer registration, adding bus details, booking tickets, cancelling bookings, and retrieving data from the database. Java provides features like object-oriented programming, exception handling, and strong memory management, which help in building a reliable and efficient system.

3.2 Framework / Libraries

3.2.1 Introduction to Swing

Swing is a part of Java that is used to create graphical user interfaces (GUI). It provides a set of components such as buttons, labels, text fields, tables, and frames to design user-friendly applications.

• Login Window
• Customer Registration Form
• Bus Details Display Window
• Ticket Booking Window
• Booking Details View.

3.3 Database Technology

The Bus Ticket Booking System uses a relational database for storing and managing data such as user details, bus information, and booking records in a structured manner The database ensures efficient data storage, retrieval, and manipulation, which is essential for maintaining accuracy and consistency in the system. In this project, Oracle Database is used as the backend, and JDBC is used for connecting the Java application to the database.

3.3.1 JDBC (Java Database Connectivity)

JDBC is an API (Application Programming Interface) used in Java to connect and interact with databases. It allows Java applications to execute SQL queries and retrieve or update data in the database. In the Bus Ticket Booking System, JDBC is used to connect the Java application with the Oracle database. It performs operations such as:

• Establishing a connection to the database
• Executing SQL queries (INSERT, UPDATE, DELETE, SELECT)
• Retrieving data from the database
• Closing the database connection

JDBC acts as a bridge between the application and the database, ensuring smooth communication and data transfer. It helps in maintaining data consistency and enables efficient handling of booking and user data.

3.3.2 Oracle Database

Oracle Database is a powerful relational database management system used for storing and managing structured data. It provides high performance, reliability, and security for handling large volumes of data. In the Bus Ticket Booking System, Oracle Database is used to store important details such as user information, bus details, and booking records. It supports SQL for data manipulation and ensures data integrity through constraints like primary keys, foreign keys, and proper data types.

3.4 Tools & IDEs

3.4.1 Visual Studio Code

Visual Studio Code (VS Code) is used as the development environment for the Bus Ticket Booking System. It is a lightweight and powerful code editor that supports Java programming through extensions.

VS Code provides various features such as syntax highlighting, code completion, debugging support, and integrated terminal, which help in writing and testing code efficiently. It also supports extensions for Java development, making it easier to compile and run Java programs.

4. DATABASE DESIGN

4.1 Database Tables

The Bus Ticket Booking System uses a relational database to store user, bus, and book information in a structured manner. The database consists of three main tables: Users, Bus, Booking.

• The Users table stores details of registered users including admin and customers.
• The Bus table stores details of available buses such as routes, seats, and ticket prices.
• The Booking table stores ticket reservation information and is linked to both Users and Bus tables.

4.2 Table Structure

4.2.1 Users Table

Field Name	Data Type	Description	Constraint
user_id	NUMBER	User ID	Primary Key
user_name	VARCHAR2(50)	Username	Not Nul
password	VARCHAR2(50)	Password	Not Nul
email	VARCHAR2(100)	Email Address	-
phone	VARCHAR2(15)	Contact Number	-
role	VARCHAR2(10)	Role (Admin/Customer)	Not Nul

4.2.2 Bus Table

Field Name	Data Type	Description	Constraint
bus_id	NUMBER	Bus ID	Primary Key
bus_name	VARCHAR2(50)	Bus Name	Not Null
source	VARCHAR2(50)	Source Location	Not Null
destination	VARCHAR2(20)	Destination	Not Null
total_seats	NUMBER	Total Seats	_
total_seats	NUMBER	Ticket Price	_

Table 2: Bus Relational Database Table

4.2.3 Booking Table

Field Name	Data Type	Description	Constraint
book_id	NUMBER	Booking ID	Primary Key
bus_id	NUMBER	Bus ID	Foreign Key
user_id	NUMBER	User ID	Foreign Key
travel_date	DATE	Travel Date	Not Null
seat_number	NUMBER	Seat Number	Not Null
total_amount	NUMBER	Total Amount	-

Table 3: Booking Relational Database Table

4.3 Keys and Constraints

Keys and constraints are used to maintain data integrity and ensure accurate data storage.

• **Primary Keys:**

o user_id in Users table

o bus_id in Bus table

o book_id in Booking table

• **Foreign Key:**

o bus_id in the Booking table references bus_id in the Bus table

o user_id in the Booking table references user_id in the Users table

• **NOT NULL Constraint**:

o Important fields such as username, password, bus details, and travel date cannot be left

• **Data Type Constraints:**

o Each field accepts only valid data types to ensure correctnessThese constraints help in maintaining consistency and avoiding duplicate or invalid data.

4.4 Table Relationships

The Users, Bus, and Booking tables are related through foreign keys.

• Each booking is associated with one user
• Each booking is associated with one bus
• One user can have multiple bookings
• One bus can have multiple bookings

This represents a **one-to-many relationship** between:

- Users and Booking
- Bus and Booking

5. SYSTEM DESIGN

5.1 System Architecture

The Bus Ticket Booking System follows a three-layer architecture:

1. Presentation Layer (User Interface)

Developed using Java (Console-based interface)

Allows users to interact with the system

2. Application Layer (Business Logic)

Processes user inputs

Performs operations like login, booking, cancellation, viewing, and updating records

3. Data Layer (Database)

o Stores user, bus, and booking data

o Managed using Oracle Database

This architecture ensures better organization, security, and maintainability.

5.2 Modules Description

The system is divided into the following modules:

1. User Management Module

• Handles customer registration

• Manages login for admin and customers

• Stores and retrieves user details

2. Bus Management Module

• Adds new bus details

• Displays available buses

• Manages routes and pricing

3. Booking Module

• Books tickets for customers

• Checks seat availability

• Calculates ticket amount

• Stores booking details

4. Cancellation Module

• Cancels booked tickets

• Deletes booking records from database

5. Database Connectivity Module

• Connects Java application with Oracle DB

• Executes SQL queries

• Retrieves and updates data

6. User Interface Module

• Provides console-based interface

• Displays menus and outputs

• Handles user interactions

6.SYSTEM IMPLEMENTATION

LOGIN MENU

Users enter username and password.
System verifies credentials using database

Admin Login: Admin enters username and password to access admin functions like adding buses, viewing bookings, and managing users.

Customer Login: Existing customers enter credentials to access booking and view their tickets.

Customer Registration: New customers can register with basic information like name, email, phone, and password.

Exit: Close the program.

```
===== LOGIN MENU =====
1. Admin Login
2. Customer Login
3. Customer Registration
4. Exit
```

Figure 1:login menu

ADMIN MENU

View Buses: Admin can see all available buses, including bus ID, name, source, destination, total seats, and ticket price.

Add Bus: Admin can add a new bus to the system, specifying all details including ticket price.

View Bookings: Admin can view all booked tickets by all users, including booking ID, bus, user, travel date, seat number, and amount.

Update Booking: Admin can update seat numbers of any booking if needed.

View Users: Admin can see all registered users along with their role (admin/customer).

Exit: Return to login menu.

```
Username: admin
Password: admin123
Admin Login Successful!

===== ADMIN MENU =====
1. View Buses
2. Add Bus
3. View Bookings
4. Update Booking
5. View Users
6. Exit
```

Figure2: admin menu

CUSTOMER MENU

View Buses: Customers can see all available buses along with details and ticket price

Book Ticket: Customers can book a seat on a bus for a specific travel date. The system checks seat availability to prevent double booking.

View My Bookings: Customers can view all their booked tickets with bus, date, seat, and amount.

Cancel Booking: Customers can cancel their booking using the booking ID **View My Details:** Customers can see their registration details like ID, name, email, and phone. **Exit:** Return to login menu.

```
2
Username: prasanthi
Password: prasanthi16
Customer Login Successful!

===== CUSTOMER MENU =====
1. View Buses
2. Book Ticket
3. View My Bookings
4. Cancel Booking
5. View My Details
6. Exit
```

Figure3: customer menu

7.CONCLUSION

The Bus Ticket Booking System has been successfully developed to manage bus reservations in an efficient and organized manner. The system replaces manual ticket booking methods with a computerized solution, thereby reducing errors, saving time, and improving data accuracy.

The application provides essential functionalities such as user registration, login authentication, viewing bus details, booking tickets, cancelling bookings, and managing records through a user-friendly interface. By using technologies like Java, JDBC, and Oracle Database, the system ensures reliable performance, secure data handling, and efficient database management.

The system also includes important features such as seat availability checking and role-based access for admin and customers, which enhance the overall functionality and usability of the application

Overall, the Bus Ticket Booking System improves the efficiency of managing ticket reservations and reduces the workload of manual processes. It provides a scalable and reliable solution that can be further enhanced with advanced features in the future.

8.FUTURE ENHANCEMENTS

The Bus Ticket Booking System developed in this project provides basic functionalities for managing bus reservations. enhancements include:

• Graphical User Interface (GUI)

The current system is console-based. It can be upgraded to a GUI-based application using Java Swing or JavaFX to make it more interactive and user-friendly.

• Online Payment Integration

Adding online payment options such as credit/debit cards, UPI, or net banking will allow users to complete ticket booking with secure transactions.

• Seat Selection System

A graphical seat layout can be implemented to allow users to visually select available seats instead of entering seat numbers manually.

• Multi-User Support

Enhancing the system to support multiple users simultaneously will make it suitable for real-time applications and larger organizations.

• Web-Based Application

The system can be converted into a web-based application using technologies like HTML, CSS, JavaScript, and backend frameworks for remote access.

• Email/SMS Notifications

Automatic notifications can be sent to users for booking confirmation, cancellation, and reminders.

• Advanced Security Features

Implementing encryption, password hashing, and role-based access control can improve system security.

These enhancements can make the system more efficient, scalable, and suitable for real-world applications.

9. REFERENCES

The following references were used during the development of the Bus Ticket Booking System:

Herbert Schildt, Java: The Complete Reference, Oracle Press, 11th Edition.

E. Balagurusamy, Programming with Java, McGraw Hill Education.

Korth, Silberschatz, Sudarshan, Database System Concepts, McGraw Hill.

- **Oracle Documentation:** https://docs.oracle.com
- **Java Documentation (Oracle):** https://docs.oracle.com/javase
- **JDBC Tutorial – Geeks for Geeks:** https://www.geeksforgeeks.org/jdbc/
- **Java Tutorial – W3Schools:** https://www.w3schools.com/java/

APPENDIX

A.1 Sample Code

Main Program: OnlineBusReservation.java

The following code represents the core implementation of the Bus Ticket Booking System. It includes functionalities such as admin login, customer registration, ticket booking, viewing bookings, and database operations using JDBC.

```
import java.sql.*;

import java.util.Scanner;

public class SimpleBooking {

    static final String URL = "jdbc:oracle:thin:@localhost:1521:XE";

    static final String USER = "system";

    static final String PASS = "12345678";

    public static void main(String[] args) throws Exception {

        Class.forName("oracle.jdbc.driver.OracleDriver");

        Connection con = DriverManager.getConnection(URL, USER, PASS);

        Scanner sc = new Scanner(System.in);

        while (true) {

            System.out.println("\n1.Insert Booking");

            System.out.println("2.View Bookings");

            System.out.println("3.Update Booking");

            System.out.println("4.Delete Booking");

            System.out.println("5.Exit");

            int ch = sc.nextInt();

            switch (ch) {
```

```
// INSERT
case 1:
    System.out.print("Booking ID: ");
    int id = sc.nextInt();
    System.out.print("Bus ID: ");
    int bus = sc.nextInt();
    System.out.print("User ID: ");
    int user = sc.nextInt();
    System.out.print("Seat No: ");
    int seat = sc.nextInt();
    PreparedStatement ps1 = con.prepareStatement(
        "INSERT INTO booking VALUES(?,?,?,SYSDATE,?,?)");
    ps1.setInt(1, id);
    ps1.setInt(2, bus);
    ps1.setInt(3, user);
    ps1.setInt(4, seat);
    ps1.setDouble(5, 500);
    ps1.executeUpdate();
    System.out.println("Inserted!");
    break;
// VIEW
case 2:
    Statement st = con.createStatement();
    ResultSet rs = st.executeQuery("SELECT * FROM booking");
    while (rs.next()) {
```

```
        System.out.println(rs.getInt(1) + " | "
                + rs.getInt(2) + " | "
                + rs.getInt(3) + " | "
                + rs.getDate(4) + " | "
                + rs.getInt(5));
    }
    break;
// UPDATE
case 3:
    System.out.print("Booking ID: ");
    int bid = sc.nextInt();
    System.out.print("New Seat: ");
    int newSeat = sc.nextInt();
    PreparedStatement ps2 = con.prepareStatement(
        "UPDATE booking SET seat_number=? WHERE book_id=?");
    ps2.setInt(1, newSeat);
    ps2.setInt(2, bid);
    ps2.executeUpdate();
    System.out.println("Updated!");
    break;
// DELETE
case 4:
    System.out.print("Booking ID: ");
    int did = sc.nextInt();
```

```
                PreparedStatement ps3 = con.prepareStatement(
                    "DELETE FROM booking WHERE book_id=?");
                ps3.setInt(1, did);
                ps3.executeUpdate();
                System.out.println("Deleted!");
                break;
            case 5:
                System.exit(0);
            }
        }
}
```

Sample SQL Queries:

Bus Query:

create table bus(bus_id NUMBER primary key,bus_name varchar2(100) NOT NULL,source varchar2(50) NOT NULL,destination varchar2(50) NOT NULL,total_seats NUMBER NOT NULL,ticket_price NUMBER(10,2) NOT NULL);

Users Query:

create table Users(user_id NUMBER primary key,user_name varchar2(100) NOT NULL,password varchar2(20) NOT NULL,email varchar2(100) UNIQUE NOT NULL,phone VARCHAR2(50),role VARCHAR2(20) CHECK (role IN ('ADMIN','CUSTOMER')) NOT NULL);

Booking Query:

```
CREATE TABLE booking(
   book_id NUMBER PRIMARY KEY,
   bus_id NUMBER NOT NULL,
   user_id NUMBER NOT NULL,
```

```
    travel_date DATE NOT NULL,
    seat_number NUMBER NOT NULL,
    total_amount DECIMAL(10,2),
    FOREIGN KEY(bus_id) REFERENCES bus(bus_id),
    FOREIGN KEY(user_id) REFERENCES users(user_id),
    CONSTRAINT unique_seat UNIQUE (bus_id, travel_date, seat_number)
);
```